C++ Standard Template Library

In C++, the Standard Template Library (STL) provides a set of programming tools to implement algorithms and data structures like vectors, lists, queues, etc.

STL implements these data structures and algorithms using general-purpose classes and functions that have been tested rigorously.

C++ STL has 3 major components:

- Containers
- Iterators
- Algorithms

C++ STL Containers

STL containers store data and organize them in a specific manner as required.

For example, **vectors** store data of the same type in a sequential order. Whereas, **maps** store data in key-value pairs.

We can classify STL containers into 3 types:

1. Sequence containers:

- Array
- Vector
- Queue
- Deque
- Forward_list
- List

2. Associative containers:

- Set
- Multiset
- Map
- Multimap

3. Derived containers**:**

- Stack

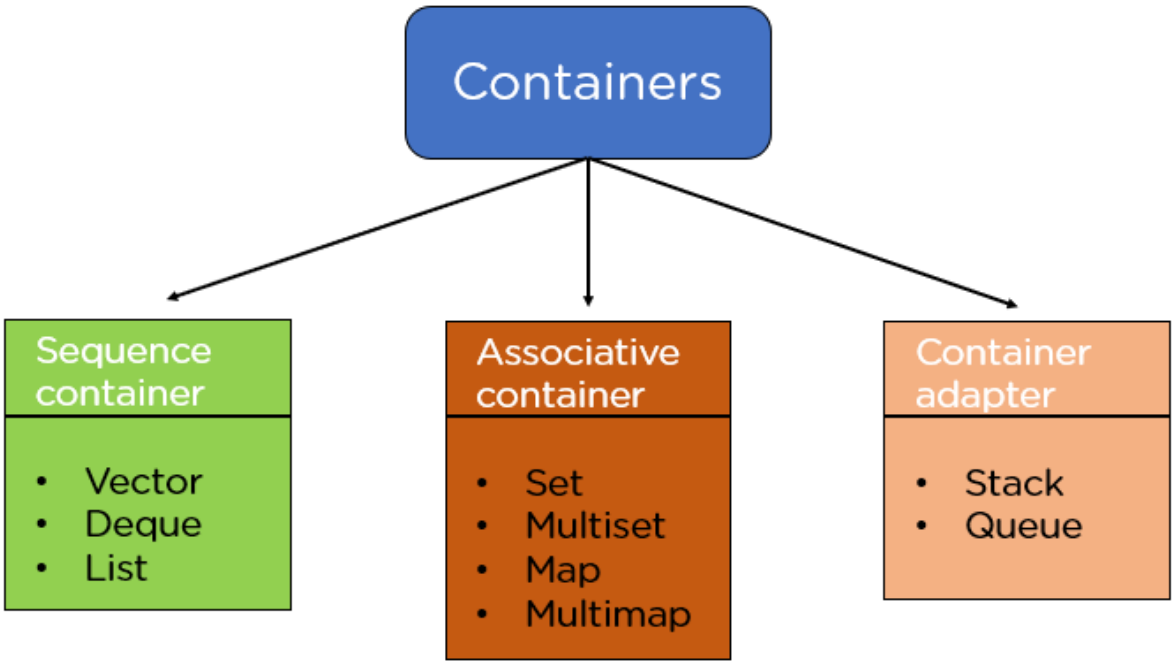
Vector India

- Queue
- Priority Queue

Containers

If you are dealing with many elements, then you need to use some sort of container. The container can be described as objects that are used to store the collection of data. It helps in recreating and implementing complex data structures efficiently.

Now, containers are further classified into three categories:



- Sequence containers: These are used to implement sequential data structures like a linked list, array, etc.
- Associative containers: These are those containers in which each element has a value that is related to a key. They are used to implement sorted data structures, for example, set, multiset, map, etc.
- Containers adapters: Container adapters can be defined as an interface used to provide functionality to the pre-existing containers.

Sequence Container:

- Vectors: Vectors can be defined as a dynamic array or an array with some additional features.

Syntax:

```
vector <int> v;
```

- Deque: Deque is also known as a double-ended queue that allows inserting and deleting from both ends; they are more efficient than vectors in case of insertion and deletion.

Syntax:

```
deque <int> d;
```

- List: List is also the sequential container and allows non-contiguous allocation. It allows insertion and deletion anywhere in the sequence.

Syntax:

```
list <int> l;
```

Associative Container:

- Set: It is an associative container that is used to store elements that are unique.

Syntax:

```
set <int> s;
```

- Multiset: This container is similar to that of the set container; the only difference is that it stores non-unique elements.

Syntax:

```
multiset <int> m;
```

- Map: Map container contains sets of key-value pairs where each key is associated with one value.

Syntax:

```
map <int, int> mm;
```

Here, the int is the key type, and the int is the value type.

Vector India

- Multimap: These containers also store key-value pairs, but unlike maps, they can have duplicate elements.

Syntax:

```
multimap <int, int> mm1;
```

Container Adapter:

- Stack: Stack follows the last-in, first-out(LIFO) approach that means new elements are added at the end and removed from that end only.
- Queue: Queue follows the first-in, first-out(FIFO) approach, which means new elements are added from the end and removed from the front.

Example:

Vector India

```cpp
#include<iostream>
#include <vector>
using namespace std;

int main()
{
    vector<int> v;

    for (int i = 1; i <= 10; i++)
    {
        v.push_back(i);
    }
    cout << "Size : " << v.size();

    v.resize(7);

    cout << "\nAfter resizing it becomes : " << v.size();

    if (v.empty() == false)
     {
        cout << "\nNot empty";
    }
    else{
        cout << "\nVector is empty";
    }

    cout << "\nElements of vector : ";

    for (auto it = v.begin(); it != v.end(); it++)
     {
        cout << *it << " ";
     }
    return 0;
}
```

In the above example, you are using vector functions and some other functions to do some operations. After declaring the vector v, you add the elements inside the vector using the push_back() function and with the help of the loop. After that, you are displaying the size of the vector using the size() function. Now using the resize() function, you are resizing the vector size to 7.

Using the empty() function, you check if the vector is empty; if it's false, then you will display not empty, and if it is not false, then you will display the vector is empty.

After that, you display all the vector elements using a for loop and functions like begin() and end(), pointing to the first element and the last element, respectively.

Iterators

Iterators are used to access data in the containers, and it helps in traversing through elements of containers using its functions. They can be incremented and decremented as per choice and help in the manipulation of data in the container.

Iterator functions are:

- begin(): This function points the iterator to the first element of the container.
- end(): This function points the iterator to the last element of the container.

Example:

```cpp
#include<iostream>
#include<iterator>
#include<vector>
using namespace std;
int main()
{
    vector<int> ar = { 1, 2, 3, 4, 5 };


    vector<int>::iterator ptr;

    cout << "The vector elements are : ";
    for (ptr = ar.begin(); ptr < ar.end(); ptr++)
    {
        cout << *ptr << " ";
    }
    return 0;
}
```

In the above example, you declare the vector using the initializer list and then declare iterator ptr to the vector.

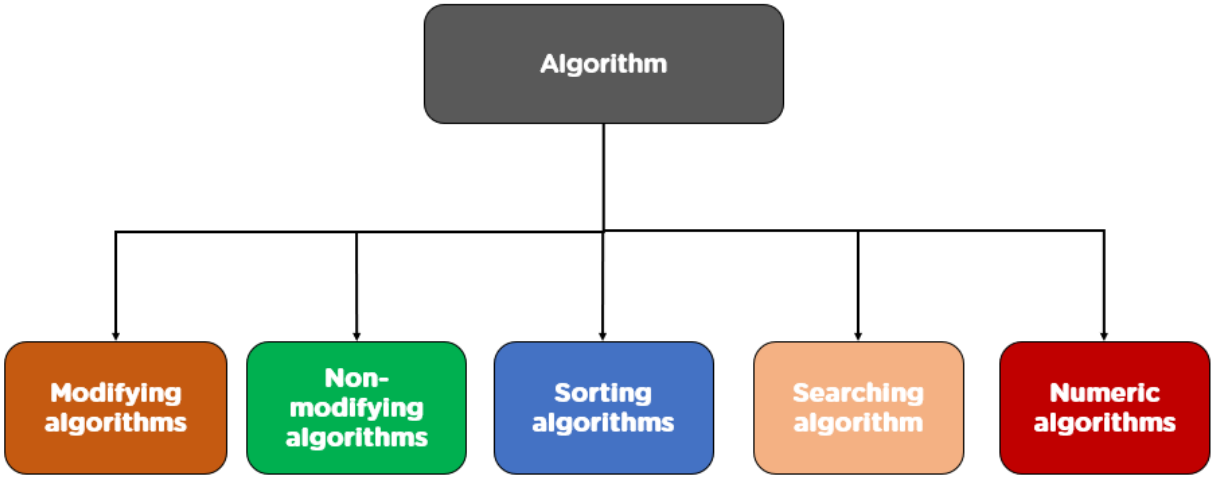
After that, you declare the vector elements using the iterator functions begin(), and end() functions inside the for loop.

Algorithms

In STL, different types of algorithms can be implemented with the help of iterators. Algorithms can be defined as functions applied to the containers and provide operation for the content of the container. for example : sort(), swap(), min(), max() etc.

Types of algorithms:

- Modifying algorithms
- Non-modifying algorithms
- Sorting algorithms
- Searching algorithms
- Numeric algorithms



Example:

```
#include<iostream>
using namespace std;

int main()
{
    int i = 7;
    int j = 10;

    cout <<"Value of i : "<<i<<endl;
    cout <<"Value of j : "<<j<<endl;

    swap(i, j);

    cout << "After swapping  i : " <<i << endl;
    cout << "After swapping  j : " <<j << endl;

    return 0;
}
```

In the above example, you have initialized the i and j variable with some values. And then, you have displayed those values; later, you have swapped those values using the swap() function.

Note :

Reference Books : Taken contents and diagrams from various websites.

Vector India